LatchMoE: Graph-Compatible Expert Offloading through Address-Stable Residency

Anonymous Author(s)

Abstract

Sparse mixture-of-experts (MoE) models reduce arithmetic per token but retain a large parameter footprint. Expert offloading can fit such models on a memory-constrained accelerator, yet conventional offloading changes weight locations on the decode path. This behavior conflicts with graph replay, which relies on stable addresses and a repeatable execution topology; disabling replay exposes a host-bound decode path.

We present LatchMoE, an expert-offloading runtime that separates dynamic expert residency from captured tensor addresses. LatchMoE preallocates fixed device slots, stages routed experts outside the replay boundary, protects slots with an explicit transfer/compute lifecycle, and executes oversized prefill working sets in capacity-bounded waves. The resulting data plane accepts placement plans without embedding a particular prediction policy.

Our implementation uses vLLM-Ascend hooks. Three graph-only starts with Qwen3-30B-A3B verify exact outputs, stable addresses, transfer ordering, bounded residency, and service release. In three matched pairs, capacity-bounded multi-wave prefill reduces repeat-median TTFT p50 and p95 by 35.21% and 22.96%, respectively, relative to graph-compatible full-layer staging. We bound this result to one model, device, memory configuration, and low-concurrency workload.

1 Introduction

Mixture-of-Experts (MoE) architectures have emerged as a prominent approach for scaling Large Language Models (LLMs) to hundreds of billions of parameters [2, 5, 7, 15]. By dynamically routing each token to only a sparse subset of experts, MoE models substantially increase model capacity while keeping per-token computation bounded [3, 14]. However, sparse activation reduces computation rather than model storage: although only a few experts are executed for a given token, the weights of all experts must remain accessible because any expert may be selected at runtime. As MoE models scale, the complete expert pool can exceed accelerator High-Bandwidth Memory (HBM) capacity. To serve such models on memory-constrained accelerators, **expert offloading** has become an important strategy: the runtime keeps a portion of expert weights in host memory and accesses or stages them on demand during inference [6, 10, 21].

To make expert offloading practical, prior systems reduce its data-movement overhead through expert caching, routing-aware prediction and prefetching, and transfer–computation overlap [1, 4, 6, 8, 21]. These techniques reduce the frequency of host-to-device transfers or hide part of their latency. However, they do not remove the execution dynamism of cache-based expert offloading: expert admission and eviction in a bounded HBM cache can change device storage assignments across inference steps.

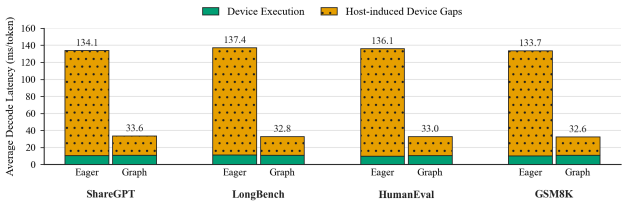

Figure 1. Average decode latency breakdown under eager execution and graph replay. Graph replay substantially reduces host-induced device gaps while leaving device execution time nearly unchanged.

Such runtime changes can be naturally accommodated under eager execution, where the host resolves the current device storage assignments and dispatches the corresponding kernels at each decoding step. This flexibility, however, comes at a substantial performance cost. Autoregressive decoding repeatedly executes a largely recurring sequence of fine-grained accelerator kernels, and launching these kernels individually incurs repeated host-side scheduling, dispatch, and synchronization overhead. **Graph replay** mitigates this overhead by capturing the recurring execution sequence once and replaying it as a unit, thereby reducing host-induced gaps between kernel executions [16, 22].

As shown in Figure 1, graph replay reduces average decode latency by approximately 75% across four workloads, from 133.7–137.4 ms/token under eager execution to 32.6–33.6 ms/token, while device execution time remains nearly unchanged. This result indicates that most of the improvement comes from eliminating host-induced device idle time rather than accelerating the underlying kernels.

However, graph replay requires its captured execution state to remain replay-stable. A dynamically managed expert cache can violate this requirement when expert admission or eviction changes the device storage assignment exposed to captured computation. A cache miss may therefore require updating or recapturing the graph to reflect new storage bindings, or falling back to eager execution. All three options add host work and reduce the benefit of graph replay. The resulting systems problem is to **enable**

dynamic expert residency and device storage assignment within a bounded HBM cache while preserving the graph-visible storage bindings required by graph replay.

UVA allows accelerator kernels to access pinned host-resident weights through device-visible addresses [12, 20]. When the backing allocations remain alive, the same addresses can be reused across graph replay [13]. UVA therefore addresses address stability for host-resident weights, but does not by itself provide routing-driven replacement within a bounded HBM expert cache. Thus, preserving graph replay under cache-based expert offloading remains challenging.

Our key insight is that **graph replay requires stable graph-visible storage bindings, not static expert residency**. A fixed set of HBM slots can provide those bindings while the runtime changes which logical experts occupy the slots. This separation allows device storage assignments to evolve without changing the bindings observed by replayed computation.

Realizing this abstraction raises three systems challenges. First, routing-dependent expert staging must be isolated from captured execution. Active experts are known only after routing, yet expert admission and eviction must not alter the graph-visible storage bindings or captured execution structure. Second, bounded slot storage must be updated without placing expert transfer entirely on the critical path. Changing routing decisions may require repeated expert movement, and the routed working set may exceed the available slots, making transfer–computation overlap essential for efficient execution. Third, slot reuse must remain correct under asynchronous transfer and computation. A slot cannot be reassigned while its previous contents are still in use, nor can a new assignment become visible before the staged weights are ready.

We present LatchMoE, a graph-compatible MoE offloading system that maps changing logical experts onto a fixed set of device slots. LatchMoE performs expert staging outside the replayed computation, allowing replayed kernels to access the same slots even as the resident experts change. For efficient reuse of bounded slot storage, LatchMoE organizes expert execution into waves and overlaps the staging of upcoming experts with ongoing computation. Finally, LatchMoE coordinates asynchronous transfer, computation, and slot reassignment through a version-controlled handoff protocol, ensuring that each slot is reused only when its previous use has completed and its new contents are ready. Together, these mechanisms enable dynamic expert residency and device storage assignment within a bounded HBM cache while preserving graph replay.

In summary, this paper makes the following contributions:

- We identify a cross-layer conflict between cache-based expert offloading and graph replay in memory-constrained MoE inference: routing-driven expert admission, eviction, and replacement cause device storage assignments to evolve, whereas graph replay reuses the graph-visible storage binding referenced by captured computation. From this conflict, we derive the key insight that replayability requires stable graph-visible storage rather than static expert residency.
- We design LatchMoE, a graph-compatible MoE offloading runtime that virtualizes dynamic logical experts over replay-stable device slots. LatchMoE isolates routing-dependent expert staging from captured computation, pipelines expert transfer with computation through wave-based execution, and coordinates safe slot reuse through a version-controlled handoff protocol.
- We implement LatchMoE on top of vLLM-Ascend and evaluate it under constrained HBM. On Qwen3-30B-A3B, LatchMoE preserves exact model outputs and graph replay under cache-based expert offloading. Separately, compared with graph-compatible full-layer staging, its multi-wave prefill execution reduces repeat-median TTFT p50 by 35.21%.

2 Background and Problem Formulation

2.1 MoE Inference and Cache-Based Expert Offloading

A Mixture-of-Experts (MoE) layer replaces the dense feed-forward network with a set of experts and a router. For each input token, the router selects a small subset of experts and combines their outputs according to the routing weights [2, 5, 7, 15]. We refer to an expert's model-defined identity as its *logical expert ID*. For an invocation of a given MoE layer at step t, let $\mathcal{A}_t$ denote the set of logical experts selected across all participating tokens. Because routing is input-dependent, $\mathcal{A}_t$ varies across inference steps. This active-set variability is especially pronounced during prefill, where many prompt tokens are processed together and may collectively activate substantially more experts than a typical decoding step [4].

When the complete expert pool does not fit in accelerator HBM, expert offloading places part of the expert weights in host memory. We focus on *cache-based expert offloading*, which maintains a bounded expert cache in HBM and stages non-resident experts from host memory on demand [1, 4, 6, 21]. Let $\mathcal{H}_t$ denote the experts of this layer currently resident in HBM, with $|\mathcal{H}_t| \leq C$, where C denotes the maximum number of experts that the configured cache can hold. A requested expert $e \in \mathcal{A}_t \cap \mathcal{H}_t$ is a cache hit, whereas $e \in \mathcal{A}_t \setminus \mathcal{H}_t$ incurs an expert cache miss and must be staged from host memory. If the cache is full, admitting a missing expert requires evicting or replacing an existing resident expert.

Cache-based offloading therefore makes expert residency and placement runtime-managed state. For each resident expert $e \in \mathcal{H}_t$, let $\pi_t(e)$ denote the device storage location

assigned to its weight tensors at step t. As routing decisions trigger admission, eviction, and replacement, both the resident set $\mathcal{H}_t$ and the placement mapping π_t may evolve over time. Thus, a logical expert has a fixed model identity, while its HBM residency and physical placement are dynamic.

2.2 Graph Replay and Replay-Stable Execution State

Graph replay reduces repeated host-side scheduling and kernel-launch overhead by capturing a recurring accelerator execution sequence and replaying it as a unit [16, 22]. Ordinary graph replay reuses the captured operation structure and graph-visible storage bindings. Updating or recapturing this state can accommodate binding changes, but requires additional host intervention [13]. We refer to the state required for ordinary replay as the *replay-stable execution state*, and denote its storage bindings by $\mathcal{B}$.

Replay stability does not require runtime data to remain unchanged. Values stored in preallocated graph-visible buffers may be updated between replays as long as the captured computation continues to access the same storage. For example, FlashInfer stores request-dependent scheduling metadata in preallocated device workspaces and updates its contents while preserving the workspace addresses referenced by captured kernels [22]. This example shows that graph replay can accommodate dynamic values behind stable storage bindings, whereas binding changes require graph update or recapture.

2.3 Dynamic Expert Placement Meets Graph Replay

The conflict arises when cache-based expert offloading exposes the current device storage assignment of expert weights directly to captured computation. Consider a logical expert e that is resident at two inference steps t and $t' > t$. If e is evicted after step t and later reloaded, it may occupy a different HBM location:

$$\pi_t(e) \neq \pi_{t'}(e).$$

Such placement changes are natural under a bounded expert cache, whose contents evolve with routing decisions.

If captured expert computation directly binds e to its current HBM location, a change in $\pi_t(e)$ also changes the graph-visible binding $\mathcal{B}$. Dynamic expert caching may therefore require the resident set $\mathcal{H}_t$ and placement mapping π_t to evolve, while conventional graph replay expects its graph-visible bindings $\mathcal{B}$ to remain reusable. The desired execution abstraction must decouple these two requirements:

$$(\mathcal{H}_t, \pi_t) \text{ may evolve}, \qquad \mathcal{B} \text{ remains replay-stable}.$$

Several straightforward alternatives avoid one side of this conflict only by sacrificing another system objective. Keeping all experts permanently resident would stabilize their physical bindings but defeats offloading under a bounded HBM budget. Falling back to eager execution accommodates arbitrary placement changes but forfeits the host-overhead savings of graph replay. Updating or recapturing the graph after each placement change similarly reintroduces host intervention into the repeated inference path.

Therefore, our goal is to support dynamic expert residency and storage assignment within a bounded HBM cache while preserving the graph-visible storage bindings required by ordinary replay.

Table 1. Routing pressure in a 200-request ShareGPT run with 12 managed layers and a 32-expert cache per layer. Decode statistics use a fixed 1/8 sampling rate. Even though decode activates only eight experts per sampled invocation, device storage assignments change frequently; every prefill invocation exceeds the cache capacity.

Metric	Prefill	Decode
Profiled layer invocations	2,400	38,098
Active experts, p50/p95/max	99/117/126	8/8/8
Invocations with $\lvert\mathcal{A}_t\rvert > 32$	100.00%	0.00%
Expert cache miss rate	–	20.26%
Invocations with a cache miss	–	68.25%
Invocations with an assignment update	–	67.59%

3 Motivation

Section 2 identifies a structural mismatch between cache-based expert offloading and ordinary graph replay. We next examine how often the underlying conditions arise in a realistic serving workload. Our characterization asks two questions: whether a small decode-time active set actually produces stable expert residency, and whether a bounded expert cache can hold the much larger active set observed during prefill.

We analyze the retained router and expert-cache profile from a 200-request ShareGPT mixed-chat run of Qwen3-30B-A3B in BF16 on one Ascend 910B2. The run uses tensor parallelism one, client concurrency one, 12 managed MoE layers, and a 32-expert cache for each managed layer. Prefill statistics cover all 2,400 layer invocations. Decode events are sampled once every eight layer invocations and contain 38,098 samples. Table 1 summarizes the results. The active-set measurements reflect the model's routing decisions, whereas cache misses and device storage assignment updates are specific to this cache configuration and residency policy.

3.1 Decode Repeatedly Changes Device Storage Assignments

A small instantaneous active set does not imply stable residency. Each sampled decode invocation activates eight experts, one quarter of the configured cache capacity. Nevertheless, 68.25% of sampled layer invocations contain at least one expert cache miss, and 20.26% of the requested experts miss in the cache. Moreover, 67.59% of sampled invocations

update at least one device storage assignment. Thus, capacity sufficient for one decode invocation does not stabilize assignments across invocations. Ordinary graph replay cannot directly bind logical experts to these changing device storage assignments without graph update or recapture.

This observation is deliberately scoped to the evaluated model, workload, and cache configuration. The exact miss rate depends on routing locality, cache capacity, and the residency policy. The important systems property is that decode-time routing repeatedly changes the device storage assignment even when the active set is much smaller than the cache.

3.2 Prefill Exceeds the Bounded Expert Cache

Prefill creates a different form of pressure. Its median invocation activates 99 distinct experts, while the p95 and maximum reach 117 and 126 experts, respectively. All 2,400 profiled layer invocations exceed the 32-expert cache capacity. This overflow is independent of the cache replacement policy: the experts in one invocation cannot be materialized simultaneously within the configured bound. Even an ideal partition would require at least three capacity-sized groups for 99.29% of the profiled invocations.

The runtime must therefore support two operating regimes. During decode, it must preserve replay while individual cache misses continually change expert residency. During prefill, it must execute an active set that can be several times larger than the available cache. A bounded implementation must therefore stage and compute these groups over time. Staging each group synchronously would expose transfer latency, so the execution model should retain opportunities to overlap transfer with useful computation.

3.3 Requirements for Graph-Compatible Offloading

Straightforward alternatives satisfy only part of this objective. Permanent expert residency preserves graph-visible storage bindings but violates the bounded-HBM premise. Eager offloading accommodates arbitrary assignment changes but gives up the graph-replay savings shown in Figure 1. Updating or recapturing the graph after assignment changes likewise reintroduces host intervention. Finally, exposing host-resident weights through stable addresses preserves replay but does not provide a dynamically managed HBM expert cache [12, 20].

These observations lead to three requirements. **First, binding stability:** runtime changes in device storage assignment must not change the graph-visible storage bindings used by ordinary replay. **Second, capacity-bounded execution:** the runtime must preserve the model's routing semantics when the active set exceeds the expert-cache capacity, while allowing transfer to overlap with computation. **Third, safe asynchrony:** storage cannot be reassigned until its previous consumer has finished, and a new assignment cannot be published before the corresponding expert transfer is ready. The next section presents an execution abstraction that satisfies these requirements.

4 Design

4.1 Overview

LatchMoE separates replay-stable storage from routing-dependent state changes. Figure 2 shows the resulting execution model. At initialization, a CPU-first host store retains expert weights and LatchMoE allocates fixed device slot banks and persistent logical-to-slot mapping buffers. During inference, a captured router produces the routed expert IDs. An eager staging boundary turns those IDs into a placement plan, transfers missing weights, and updates mapping values in place. A captured expert MLP then consumes the same slot and mapping allocations on every replay.

The design has three core mechanisms. Replay-stable slot virtualization decouples expert identity from physical allocation. Replay-boundary staging publishes a new mapping only after transfer readiness. A versioned lifecycle and a capacity-bounded wave executor make reuse correct when transfer and compute overlap or when the active set exceeds capacity. The placement-plan interface orders routed experts but does not embed a prediction policy in the data plane.

4.2 Replay-stable expert-slot virtualization

For each managed MoE layer, LatchMoE allocates two contiguous tensors that hold the layer's fixed set of physical W_{13} and W_2 expert slots. The first dimension indexes physical slots; each slice has the shape and layout expected by the fused expert kernel. These allocations persist across capture and replay. A second persistent tensor maps each logical expert ID to a physical slot ID. Captured computation receives the slot tensors and mapping buffer, so its graph-visible pointers do not depend on which logical experts are resident.

Each slot records a logical owner, a monotonically increasing version, a state, and its last-use step. Reassigning a slot removes the old logical owner from the resident index, installs the new owner, increments the version, and enters the loading state. A lease is the tuple of slot ID, logical expert, and version. The version distinguishes two ownership intervals that reuse the same physical slot and lets asynchronous completions reject stale work.

The mapping is valid only for ready experts. Before the captured MLP consumes routed IDs, LatchMoE remaps them through the persistent logical-to-slot tensor. The physical expert count remains the number of allocated slots rather than the number of logical experts. Thus replacement changes slot contents and mapping values, while the tensors bound into the captured computation remain fixed.

4.3 Replay-boundary staging and safe publication

The router must run before the runtime knows the active expert set, but the expert MLP must run after all selected

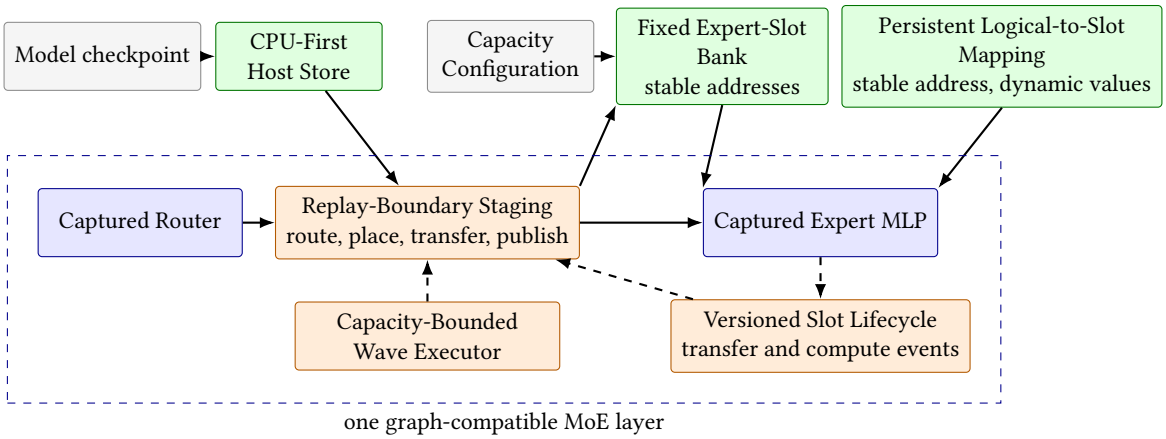

Figure 2. LatchMoE keeps graph-visible slot and mapping addresses fixed while changing their contents and values at an eager replay boundary. Solid arrows carry weights or routing data; dashed arrows carry wave and lifecycle control. Blue blocks are captured, orange blocks execute eagerly, and green blocks are stable allocations.

weights are available. LatchMoE exposes the interval between these operations as an eager staging boundary. The captured router produces routed expert IDs, the runtime deduplicates that set, and a placement plan orders the routed experts. The plan must be a permutation of the actual routed set: it may change staging order, but it cannot add an unrouted expert or omit a routed expert.

For a cache miss, the transfer engine assigns a slot version, enqueues the expert's W_{13} and W_2 copies on a dedicated H2D stream, and records a readiness event. Publication follows a fixed order. The runtime first validates that the slot still matches the lease, installs the readiness event as a dependency of the consumer stream, changes the slot to ready, and only then publishes the logical-to-slot mapping. This order prevents captured compute from observing either incomplete weights or a completion event that belongs to an older slot owner.

Hits and misses share the same output interface: ready physical slot IDs plus the persistent mapping tensor. The captured MLP therefore does not encode whether an expert was already resident, loaded on demand, or prefetched for an upcoming wave.

4.4 Versioned transfer–compute lifecycle

The slot state machine contains four externally relevant states: empty, loading, ready, and computing. Empty and ready slots may be selected for a new owner, while loading and computing slots are not reassignable. Before expert compute begins, the runtime acquires leases for the active slots and marks them computing. A completion event protects those leases until the consumer stream has finished. Release validates that owner and version still match the recorded lease before returning a slot to the ready state.

This protocol enforces two safety properties. First, transfer completion cannot make a stale owner ready because readiness checks the version created when the copy was issued. Second, eviction cannot overwrite an in-flight expert because a computing slot is excluded from reassignment. Violations raise an error instead of redirecting execution to eager mode. The fail-closed behavior makes graph compatibility part of the runtime contract rather than an unobserved best effort.

4.5 Capacity-bounded multi-wave execution

Decode commonly activates a small expert set, but a prefill request aggregates routing decisions over many tokens. The union of routed experts can exceed the slot capacity even though each token selects only a few experts. LatchMoE handles this overflow by partitioning routed token–expert pairs into waves. Each wave contains no more distinct missing experts than its staging bank can hold. Hit-only work may be scheduled separately so that a ready main-slot expert need not be copied into a temporary wave buffer.

LatchMoE executes waves without changing router assignments. It retains every routed pair's original flattened top-k position, computes wave-local expert outputs, concatenates those outputs, reconstructs one global native unpermute index, and invokes the ordinary token-combine operation once with the original top-k weights and output shape. It rejects incomplete or duplicate pair coverage. The single final combine avoids the numerical reordering introduced by independently combining and accumulating each wave.

Multiple stage banks form a software pipeline. While the current wave computes, the transfer stream may populate another bank for the next wave. The scheduler can order waves by their required transfer bytes, but it cannot change pair membership or final combination semantics. A stage

bank is reused only after its compute lease completes, so the same versioned lifecycle protects both the persistent decode slots and transient prefill slots.

4.6 Policy-neutral placement

LatchMoE defines the data-plane constraints for a placement plan: target layer, complete routed set, and a valid ordering. The default behavior preserves the router's first-seen order. An external controller may instead prioritize predicted reuse or transfer cost, but it must return a permutation of the routed experts and obey the same capacity and lifecycle checks. Separating this interface from slot management allows placement policies to share one graph-compatible execution substrate and keeps policy quality outside the correctness argument of this paper.

5 Implementation

We implement LatchMoE as a Python package that registers through vLLM's general plugin entry point. The package integrates with a hook-enabled vLLM-Ascend runtime; the upstream request scheduler and OpenAI-compatible serving API remain unchanged. The host runtime and the Ascend platform plugin are pinned as a compatible stack because the staging boundary changes the internal fused-MoE execution contract rather than the public serving interface.

5.1 Graph integration

The integration decomposes a managed MoE layer into a captured router, the custom `vllm::moe_offload_stage` splitting operation, and a captured expert MLP. The splitting operation returns control to the host after routing so that LatchMoE can stage the active experts. The subsequent graph piece reads the persistent slot weights and logical-to-slot buffer. The runtime locks their data pointers at capture and validates them before replay.

Graph-compatible runs use PIECEWISE ACLGraph. The benchmark validator requires explicit capture and replay records and rejects forced eager execution, eager fallback, graph errors, address changes, stale H2D completion, and slot-version violations. Disabling LatchMoE restores the host runtime's null hooks. The launcher also rejects simultaneous use of the native weight-offload path and LatchMoE because two independent owners of expert storage would violate the slot lifecycle.

5.2 Loading and memory management

CPU-first loading creates managed expert tensors in host memory during model initialization. The host store keeps one contiguous W_{13} tensor and one contiguous W_2 tensor per layer and exposes per-expert views to the transfer engine. It checks shape, dtype, stride, device, layer coverage, and expert coverage before releasing the original device expert banks. Host pinning is used when the runtime supports it and reports failures explicitly.

Automatic configuration derives the resident-layer selection and slot count from the requested offload amount, physical HBM, a KV-cache reserve, and a cap on the fraction of HBM assigned to slot storage. Explicit overrides remain available for controlled sensitivity experiments. The memory ledger accounts separately for the host store, persistent slots, transient prefill stage banks, mapping tensors, full-layer fallback storage, and any original weights that remain allocated.

5.3 Transfers and lifecycle enforcement

The transfer engine maintains a dedicated H2D stream. It validates layout compatibility before copying and batches consecutive host and slot slices when their storage is contiguous. Each asynchronous load returns a readiness object that contains the NPU event and the exact slot leases made consumable by that event. The consumer installs the event dependency before the object marks the leases ready. The runtime then publishes mapping values.

Compute protection uses an event recorded after the expert operation. Slots remain in the computing state until the event completes and every recorded lease still matches its current owner and version. Shutdown drains pending transfer and compute work, closes profiling writers, releases managed storage, and emits a release acknowledgement for the benchmark harness.

5.4 Wave execution and profiling

The prefill path supports a configurable number of stage banks and prefetch depth. The qualified configuration uses two stage banks and depth one. Wave plans carry the original pair offsets needed for one native recombination, and strict validation rejects duplicate or missing coverage. A recoverable preflight or native-recombination qualification failure may select the explicit full-layer overflow mode; NPU, ACL, memory, address, and lifecycle failures are not swallowed by that path.

Profiling records JSONL events for layer registration, slot ownership, transfers, mapping publication, wave staging and compute, graph replay issue, memory accounting, and service release. Benchmark manifests bind each run to the runtime revisions, model and workload digests, device, command line, environment, graph mode, and raw client/server artifacts. These records support the fail-closed checks used in Section 6.

6 Evaluation

Our current evaluation answers three questions within a deliberately narrow qualification boundary:

Q1: Replay and correctness. Does LatchMoE preserve graph-visible addresses, transfer ordering,

capacity bounds, and exact model outputs across independent service starts?

Q2: Multi-wave staging. Under a matched graph-compatible contract, how does capacity-bounded multi-wave prefill compare with blocking full-layer staging?

Q3: Mechanism exercise. Does the online runtime issue next-wave transfers before current-wave compute, preserve native recombination, and account for its bounded storage?

These questions validate the runtime mechanism and one matched prefill result. They do not establish performance breadth across models, devices, memory budgets, workloads, or serving concurrency.

6.1 Experimental setup

All reported graph-only experiments use Qwen3-30B-A3B in BF16 on one physical Ascend 910B2 with tensor parallelism one. LatchMoE manages 12 MoE layers with a 14-GiB host-offload target and 32 physical slots. The server uses `max_num_seqs=1`; client concurrency is one and prefix caching is disabled. Requests are drawn from a fixed ShareGPT manifest. The matched multi-wave experiment uses the same ordered 200-request manifest in every unit and generates 128 output tokens per request.

Every formal unit starts a fresh managed service, requires PIECEWISE ACLGraph capture and replay, retains raw client and server artifacts, and ends with a release acknowledgement and a physical HBM sample. Runs with eager fallback, forced eager execution, NPU/ACL errors, OOM, address changes, stale transfer events, or slot-generation violations fail validation. Exact output comparison uses request IDs and complete output-token arrays, not decoded string equality.

6.2 Q1: Replay-stable execution and exactness

Table 2 summarizes three independent formal starts from the graph-lifecycle qualification. Each start serves 11 requests and 1,408 output tokens. The first repeat is compared with the preceding short gate, and later repeats are compared with the prior repeat. Across the three formal starts, all 33 requests and all 4,224 output tokens match exactly.

The runtime-level gates also pass in every start. All 12 managed layers retain the slot-bank and logical-map pointers recorded at capture. Every checked compute lease preserves its owner and version until completion. Every recorded decode miss installs the consumer dependency and reaches readiness before the logical mapping is published. No multi-wave active set exceeds the 32-slot capacity, and every service emits a release acknowledgement before physical HBM returns to 5%. These observations establish the replay and lifecycle contract for the evaluated configuration.

Table 2. LatchMoE preserves exact outputs and graph/lifecycle invariants across three independent service starts. TTFT and TPOT are qualification measurements, not a matched performance comparison.

Metric	Run 1	Run 2	Run 3
Successful requests	11/11	11/11	11/11
Exact output tokens	1,408	1,408	1,408
TTFT p50 (ms)	573.64	537.70	573.11
TPOT p50 (ms/token)	55.33	55.93	53.56
Peak/final HBM (%)	91/5	91/5	91/5
Graph fallback events	0	0	0

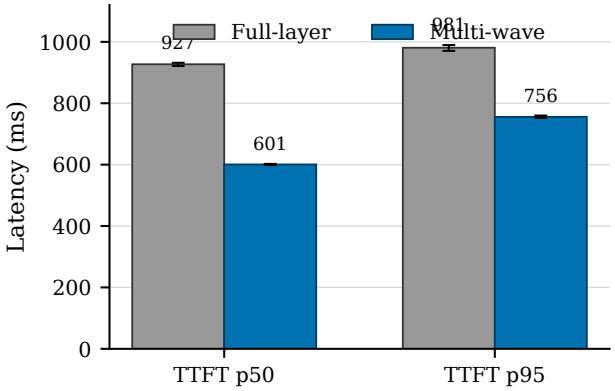

Figure 3. Capacity-bounded multi-wave staging reduces the repeat-median TTFT p50 by 35.21% and TTFT p95 by 22.96% relative to graph-compatible full-layer staging. Bars show the median of three independent service-start summaries; whiskers show their minimum and maximum. The comparison uses one Ascend 910B2, Qwen3-30B-A3B BF16, a 14-GiB offload target, 32 slots, and concurrency one.

Table 3. Matched full-layer and multi-wave results. Each arm aggregates three fresh starts and 600 ShareGPT requests. Latency entries are the median of the three repeat-level quantiles.

Mode	TTFT p50 (ms)	TTFT p95 (ms)	TPOT p50 (ms/token)
Full-layer	927.06	980.89	54.33
Multi-wave	600.65	755.68	54.42

6.3 Q2: Matched multi-wave prefill

We compare *full-layer* staging, which materializes the layer's expert working set before compute, with *multi-wave* staging, which executes the same routed work through bounded stage banks. The two arms differ only in the overflow-mode setting. We use a fixed AB/BA/AB order to form three matched pairs, with one fresh service start per unit.

As Figure 3 and Table 3 show, the repeat-median TTFT p50 falls from 927.06 to 600.65 ms, a 35.21% reduction. The corresponding TTFT p95 falls from 980.89 to 755.68 ms, a 22.96% reduction. Pair-level p50 reductions range from 35.02% to 35.55%, and pair-level p95 reductions range from 21.71% to 23.63%.

TPOT does not show the same reduction. Its repeat-level p50 values are 53.87/54.74/54.33 ms/token for full-layer and 55.34/54.42/53.87 ms/token for multi-wave. The supported performance conclusion is therefore limited to time to first token in this prefill-heavy boundary, not a decode-speedup claim. Each arm completes 600/600 requests and 76,800 output tokens. All six units produce identical token arrays, capture and replay the graph, record zero fallback and runtime error events, release their managed service, and return to 5% final HBM.

6.4 Q3: Overlap, recombination, and memory bounds

The multi-wave stability profile contains 2,544 prefill-layer events and 9,077 waves. Every wave is issued before its compute begins. The runtime submits 6,533 next-wave prefetches before the current wave's compute and records no late after-compute prefetch issue. Aggregate stage wait is 1,593.87 ms across the profile, compared with 7,493.41 ms of profiled MLP time, and the maximum per-layer stage wait is 1.27 ms. These measurements show that the intended software-pipeline schedule executes online; they do not isolate the causal latency contribution of overlap.

Exactness is checked separately from scheduling. Eleven trigger requests match the full-layer oracle for serial eager multi-wave, overlapped eager multi-wave, and overlapped graph multi-wave execution, covering 1,408 token IDs per mode. A tensor-level check over 136 BF16 routed pairs split across five interleaved waves is bitwise equal to the native combine. The only online combine mode in the qualified profile is the native-recombination path.

The final memory ledger reports 13.50 GiB in the host expert store, 3.38 GiB in persistent device slots, and 0.56 GiB in two prefill stage banks. Total device slot and stage storage is 3.94 GiB. All 12 original expert-weight banks are released, no wave exceeds 32 slots, and the physical 65,536-MiB device peaks at 91% HBM without an OOM or memory-growth failure.

7 Discussion and Limitations

7.1 Stable storage is the reusable interface

LatchMoE's main lesson is that dynamic object identity and graph replay are not inherently incompatible. The conflict arises when identity changes are exposed as changes to graph-visible allocations or execution structure. A fixed storage interface moves that dynamism into contents and metadata that can be updated at a controlled boundary. The same principle may apply to other captured operators with a bounded, replaceable working set, provided that the runtime can identify a legal staging boundary and enforce publication and reuse ordering. Our experiments establish this lesson only for the implemented MoE path.

The versioned lease is equally important as the fixed allocation. Stable pointers alone do not prevent a transfer completion for an old owner from publishing stale contents, nor do they prevent eviction during in-flight compute. Address stability supplies replay compatibility; owner/version checks and stream dependencies supply temporal correctness. Treating these as one contract makes failures observable and testable.

7.2 Control policy is outside the data plane

LatchMoE accepts an ordered placement plan but constrains it to the actual routed expert set. This boundary lets future caching or prediction policies reuse the fixed slots, transfer engine, wave executor, and graph checks. The current evaluation does not compare placement policies and should not be read as evidence that the default ordering is optimal. Such a study requires a shared trace, identical storage capacity, and policy-specific hit, transfer, and latency measurements on the same data plane.

7.3 Scope of the current evidence

The online evidence covers one unquantized model, one Ascend 910B2, tensor parallelism one, a 14-GiB offload target, 32 slots, `max_num_seqs=1`, client concurrency one, and disabled prefix caching. It validates exactness, graph replay, lifecycle ordering, bounded residency, and service release in that configuration. The matched result further shows lower TTFT for multi-wave than full-layer staging under the same narrow contract. It does not show a general TPOT improvement.

Several dimensions remain open. Higher concurrency changes the union of active experts and may alter both slot pressure and transfer–compute overlap. A different model can change expert shapes, routing distributions, and the number of managed layers. Other slot counts and memory budgets can change wave count and KV-cache headroom. Prefix-cache reuse is disabled because the current correctness contract does not cover cache hits across the offload staging boundary. Multi-device expert parallelism introduces device-to-device communication and is not represented by the single-device host-to-device path.

Before making a broad serving claim, the evaluation must compare full residency, native prefetch offload, the legacy layered path, and LatchMoE under the same graph-only manifests. It must also add model, capacity, workload, and concurrency sensitivity, retain graph or capacity failures as results, and report dispersion across independent starts. Until that matrix is complete, the paper's evidence supports a graph-compatible mechanism and the scoped multi-wave TTFT result.

8 Related Work

Sparse MoE inference. Sparsely gated MoE models increase parameter capacity while activating a subset of experts for each token [3, 5, 14, 15]. Systems for large MoE models also combine routing with model and expert parallelism [11, 14]. LatchMoE does not change the model, router, or expert computation. It addresses the runtime storage interface used when expert weights cannot remain fully resident on one accelerator.

Expert offloading and scheduling. Expert-offloading systems reduce host-to-device movement or its exposed cost through caching, prediction, prefetching, mixed precision, fine-grained placement, and pipelined execution [1, 4, 6, 8, 10, 18, 19, 21]. FlexGen studies coordinated placement and execution across heterogeneous memory for dense-model inference [17]. These directions decide which weights to retain or move and when to move them. LatchMoE is complementary: its placement-plan interface can accept such decisions, while its contribution is the fixed physical slot, persistent mapping, and lifecycle abstraction that keeps replacement compatible with captured MoE execution.

Graph-based inference execution. Nimble compiles dynamic neural-network execution, while FlashInfer provides graph-oriented mechanisms for customizable LLM inference [16, 22]. vLLM reduces serving overhead through continuous batching and paged KV-cache management [9]. LatchMoE focuses on a different mutable resource: expert weights. It inserts an explicit eager boundary between captured routing and captured expert computation, then keeps the downstream weight and mapping allocations stable while their contents change.

Positioning. Prior offloading work primarily optimizes the dynamic placement decision; graph-oriented runtimes primarily optimize reuse of repeated execution. LatchMoE connects these concerns by defining which state must remain stable and which state may change between captured regions. It leaves policy quality to the former line of work and preserves the replay interface required by the latter.

9 Conclusion

LatchMoE decouples dynamic logical expert residency from the stable physical storage required by graph replay. Fixed expert slots and persistent mappings form the replay-visible interface; an eager staging boundary updates their contents; versioned transfer and compute dependencies make reuse safe; and capacity-bounded waves execute oversized prefill working sets with one native recombination. In the qualified single-NPU configuration, three independent starts preserve exact outputs, stable addresses, transfer ordering, bounded residency, graph replay, and resource release. A matched six-unit experiment shows that multi-wave staging reduces repeat-median TTFT p50 by 35.21% and TTFT p95 by 22.96% relative to full-layer staging, while TPOT remains comparable. These results support the replay-stable storage abstraction and its scoped prefill benefit; broader serving claims require the remaining baseline and sensitivity matrix.

References

[1] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. 2025. MoE-Lightning: High-Throughput MoE Inference on Memory-constrained GPUs. In *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. https://doi.org/10.1145/3669940.3707267

[2] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. *arXiv preprint arXiv:2401.06066* (2024).

[3] Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten P. Bosma, Zongwei Zhou, Tao Wang, Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V. Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. 2022. GLaM: Efficient Scaling of Language Models with Mixture-of-Experts. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. 5547–5569.

[4] Zhiyuan Fang, Yuegui Huang, Zicong Hong, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. 2025. Klotski: Efficient Mixture-of-Expert Inference via Expert-Aware Multi-Batch Pipeline. In *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 574–588. https://doi.org/10.1145/3676641.3716261

[5] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.

[6] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated MoE: An Algorithm-System Co-Design for Fast and Scalable Mixture-of-Expert Inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*. 1018–1031.

[7] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of Experts. *arXiv preprint arXiv:2401.04088* (2024).

[8] Rui Kong, Yuanchun Li, Qingtian Feng, Weijun Wang, Xiaozhou Ye, Ye Ouyang, Linghe Kong, and Yunxin Liu. 2024. SwapMoE: Serving Off-the-shelf MoE-based Large Language Models with Tunable Memory Budget. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 6710–6720. https://doi.org/10.18653/v1/2024.acl-long.363

[9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. https://doi.org/10.1145/3600006.3613165

[10] Han Li, Jingwei Sun, Junqing Lin, and Guangzhong Sun. 2026. CommitMoE: Efficient Fallback-Free MoE Inference with Offloading Under GPU Memory Constraints. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 22904–22912. https://doi.org/10.1609/aaai.v40i27.39454

[11] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. 2023. Accelerating Distributed MoE Training and Inference with Lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association.

[12] NVIDIA Corporation. 2026. CUDA C++ Best Practices Guide. NVIDIA CUDA Documentation. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/ Accessed: 2026-08-18.

[13] NVIDIA Corporation. 2026. PyTorch CUDA Graph Integration. CUDA Graph Best Practice for PyTorch. https://docs.nvidia.com/dl-cuda-graph/latest/torch-cuda-graph/torch-integration.html Accessed: 2026-08-18.

[14] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. 18332–18346.

[15] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations (ICLR)*.

[16] Haichen Shen, Jared Roesch, Zhi Chen, Wei Chen, Yong Wu, Mu Li, Vin Sharma, Zachary Tatlock, and Yida Wang. 2021. Nimble: Efficiently Compiling Dynamic Neural Networks for Model Inference. In *Proceedings of Machine Learning and Systems*.

[17] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning*.

[18] Xiaoniu Song, Zihang Zhong, Rong Chen, and Haibo Chen. 2024. ProMoE: Fast MoE-Based LLM Serving Using Proactive Caching. *arXiv preprint arXiv:2410.22134* (2024).

[19] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2024. HOBBIT: A Mixed Precision Expert Offloading System for Fast MoE Inference. *arXiv preprint arXiv:2411.01433* (2024).

[20] vLLM Project. 2026. UVA Offloader. vLLM Documentation. https://docs.vllm.ai/en/stable/api/vllm/model_executor/offloader/uva/ Accessed: 2026-08-18.

[21] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh K. Marina. 2024. MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache. *arXiv preprint arXiv:2401.14361* (2024).

[22] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie W. Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. *arXiv preprint arXiv:2501.01005* (2025). https://doi.org/10.48550/arXiv.2501.01005